POLITECNICO
MILANO 1863

AN2DL - First Homework Report
Gradient Ascent

Edoardo Fortuna, Massimo Bruscaglioni, Alberto Cantele,

edoardofortuna, massibrusca, albertocantele,

233817, 251705, 259056,

November 24, 2024

1 Introduction

1.1 Problem statement

The goal of this project is to develop a Convolutional Neural Network (CNN) to automatically classify **96x96 RGB images of red blood cells (RBC)**, which can be subdivided into **8 different classes**.

1.2 Objectives

The main goal is to classify in the "best" way possible the RBCs images of an unknown dataset. In this case, the model is evaluated on the **accuracy** metric, defined as:

$$accuracy = \frac{\sum (y == \hat{y})}{len(y)} \tag{1}$$

,where: y is the set of real labels, and $\hat{y}$ is the set of our predictions.

1.3 Approach

The first task we performed is **dataset inspection**, followed by **pre-processing** in order to get images that are suitable for training. The next step was the initial design of the network, here called *base model*, which was used to determine the baseline performance. The network is then being trained on the processed dataset, which has also been split into **training-validation** and **test sets**. Finally, the model has been evaluated, first on the validation set, in order to gather feedback for the choice of the architecture and hyper-parameters, then on the test set, to evaluate the model's performance, based on some appropriate metrics. The hyperparameters of the network have been manually updated at each experiment.

2 Problem Analysis

2.1 Dataset characteristics

The dataset we were given is composed of 13759 images. We noticed that there was some **class imbalance**, in fact some classes (e.g. Neutrophil, Eosinophil) had significantly more images than others (e.g. Basophil, Lymphocyte). The image per class distribution, prior to outliers and potential duplicates removal, is shown in Figure 1.

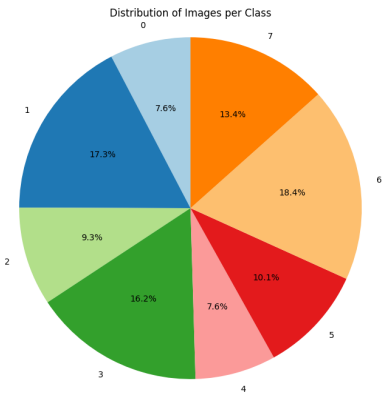

Figure 1: Class distribution(prior to outliers and potential duplicates removal). In order: Basophil, Eosinophil, Erythroblast, Immature granulocytes, Lymphocyte, Monocyte, Neutrophil, Platelet

2.2 Main challenges

This project presented many different challenges, some of them were expected, others were discovered as we progressed. The most significant one was the hyperparameters **tuning process**, and creating an architecture that was able to balance both **accuracy, generalization ability and computational efficiency**, as we did not have access to powerful GPUs/TPUs. Another very important aspect that we worked on was the **quality of the data**, that we improved through pre-processing. As we were dealing with a multi-class classification problem, the **class imbalance** issue needed to be taken into account as well.

2.3 Initial assumptions

We initially assumed that the images of the dataset on which the classifier would be evaluated would be similar to the ones of the dataset we were given. Thus, it seemed reasonable to tune the network's hyperparameters based on our local prediction quality. However, we recognize that variations may exist in the actual images to be classified, making it essential to prioritize generalization. Additionally, it was assumed that the images in our dataset might need some cleaning and pre-processing. Finally, we were aware of the necessity of a good **data exploration** in order to identify important features on the images that could have helped us to understand which were the most important parameters to modify on the network.

3 Method

Before diving into the design of the network, the dataset has been inspected manually. Some outliers were found and removed, along with all the potential duplicates. Pre-processing included pixel normalization ([0,1] range) and dataset splitting (70% training, 15% validation, 15% test) to preserve sufficient training data. The base model (V0) was a 2-layer CNN with 3x3 kernels (32, 64 filters), ReLU activation, and 2x2 Max-Pooling for dimensionality reduction. It was followed by a dense layer with 16 neurons and a softmax output layer (8 neurons). Trained with the Adam optimizer and categorical cross-entropy loss, the model took 30-50 epochs of training with early stopping. **Mixed precision** was used to improve training efficiency[1]. In V1, we expanded to a 3- layer CNN (8, 16, 32 filters), increased the dense layer to 64 neurons, and added a 0.5 dropout to reduce overfitting. Class-balanced weights addressed data imbalance. **L2 regularization** (0.001) and batch normalization were added to improve robustness, but accuracy gains were minimal. For V2, we replaced Flatten with **Global Average Pooling (GAP)**. It has been shown[2] that GAP leads to smaller testing error even in *conventional* CNNs. Also we increased filters to 16, 32, 64, but reverted to the original configuration due to negligible accuracy improvement. In V3, image augmentation (*ImageDataGenerator*) was applied to enhance generalization by introducing random distortions (rotation, shear, translation). We then tried out *AugMix* to introduce stronger distorsion. Finally, *MobileNetV2* was fine-tuned as a pretrained model with frozen lower layers, a low learning rate for higher layers, and a dense layer with 64 neurons and 0.5 dropout rate. However, local accuracy decreased, likely due to augmentation challenges.

4 Experiments

All the experiments were made across the 5 models presented in Table 1: We decided to develop different versions of the same network, except for the V4 (*MobileNetV2*), to better understand how different techniques affect accuracy and train speed.

5 Results

Developing different networks made us observe that:

- Implementing image normalization after a proper pre-processing boosted up accuracy, because avoid instable gradients and increase optimization efficiency. **Balancing class weights** improved significantly the quality of our predictions, while the **data augmentation and the reduction of model complexity** helped with Codabench score. Also balancing class weights improved significantly the quality of our predictions, while the data augmentation and the reduction of model complexity helped with codabench score

- Passing from a more complex network to a lighter one significantly improved our accuracy performance on codabench test set(from 7% to 26%), leading us to think a more complex one probably would have led to overfitting

- Unexpectedly, even boosting up data augmentation and increasing generalization we still weren't able to achieve good scores on codabench

6 Discussion

- Strengths and weaknesses: We developed a network aiming to optimize both generalization through data augmentation techniques and the use of regularization techniques such as L2, batch normalization, early stopping and dropout; and training speed, by choosing low complexity configurations in order to maintain a light structure. We think the poor results were due to an overfitting problem: since we evaluated the performance mainly on local sets we think our network specialized on our data, but still gave good results on local test due to the high similarity between train and test, limiting the generalization capabilities in respect to external data. Trying different techniques of data augmentation still led little to no results. We suspect the main limitation of our model is on generalization capabilities, leading to a good network for the analysis of clean or slightly distorted RBC images, but not robust enough to understand complex distortions such as the ones we assume to be introduced on Codabench test set.

- Limitations and assumptions: We assumed the set on Codabench was strongly distorted therefore we tried different data augmentation techniques still retrieving no results. Our main limitation was the scarcity of data and the different domain in respect to the test set on Codabench since we had no information about the kind of distortions applied to those images.

7 Conclusions

To sum up:

- Edoardo and Alberto mainly focused on implementing the code and testing the networks. Massimo's contribution included the design of the experiments and data collection. Everyone contributed to the analysis of the results and the decisional process behind the testing.

- V3 is the model with the best performance on codabench

- Some improvements may be the introduction of deeper or wider layers in combination with *AugMix*, or the possibility to develope a dynamic uploader for *Augmix*, in order to change the augmentation applied to the images after each epoch such as with *Image datagen*. Another solution might be to change approach and realize N different networks, each with a responsibility factor for each class. Through a weighted voting mechanism based on the responsibility factor of the predicted class for each network. The weights of each classifier would be updated depending on their responsibility factor for the class they predicted.The responsibility factors would be iteratively adjusted based on how well each network performs in predicting each specific class. In this way each network would specify in the prediction of some classes resembling the activity of ML ensemble classifiers.

- Future work could involve experimenting with different optimization algorithms. Another possibility is applying transfer learning and/or

fine-tuning on architectures different than *MobileNet*. Additionally, exploring *Neural Architecture Search* may yield better network designs.

References

[1] Keras. Mixed precision. `https://keras.io/api/mixed_precision/`.

[2] S. Y. Min Lin, Qiang Chen. Network in network. `https://arxiv.org/pdf/1312.4400v3`.

Table 1:
Comparison of the best performance models in term of accuracy and F1 score, local accuracy was used (from local tests and not from Codabench submissions)

Model	Local Accuracy	F1 Score	Architecture
Base model (V0)	82.2 ± 2.4	81.1 ± 2.8	2 Conv Layers + Max Pool layers + Flatten layer + Dense layers
Model (V1)	86.2 ± 2.0	84.5 ± 1.8	V0 + Batch Normalization + GAP + L2 regularization
Model (V2)	93.3 ± 1.8	91.8 ± 1.5	V1 + Image datagen augmentation + class balance + Flatten
Model (V3)	89.8 ± 1.6	87.4 ± 1.5	V1 + augmix augmentation + class balance + Flatten
Model (V4)	80.5 ± 1.8	78.4 ± 2.0	MobileNetV2 pre-trained NN + Augmentation + class balance